

Лекція

Методи колективного обміну
повідомленнями в MPI та їх
застосування для матричних
обчислень

Методи колективної взаємодії процесів

- Синхронізації
- Передачі даних
 - один-до-багатьох,
 - багато-до одного,
 - багато-до-багатьох
- Блокуючі і неблокуючі

Метод MPI_Barrier

- `int MPI_Barrier(MPI_Comm comm)`
- примушує очікувати завершення виконання операцій (які передують цьому виклику) усіма процесами, що належать до вказаного комунікатора
- таким чином, синхронізує перехід усіх процесів до наступної інструкції програми

Метод MPI_Scatter

```
MPI_Scatter( void *sendbuf,  
            int sendcnt,  
            MPI_Datatype sendtype,  
            void *recvbuf,  
            int recvcnt,  
            MPI_Datatype recvtype,  
            int root, /* процес-відправник */  
            MPI_Comm comm);
```

- Для виконання розсилки даних метод потрібно викликати в кожному процесі з однаковими значеннями `root` та `comm` та відповідними значеннями `recvcnt` та `recvtype`. Аргументи `sendbuf`, `sendcnt`, `sendtype` враховуються тільки процесом-відправником даних.
- Перші три аргументи у процесах, що приймають дані, ігноруються, а кількість та тип отримуваних елементів, номер процесу-відправника та комунікатор збігаються.

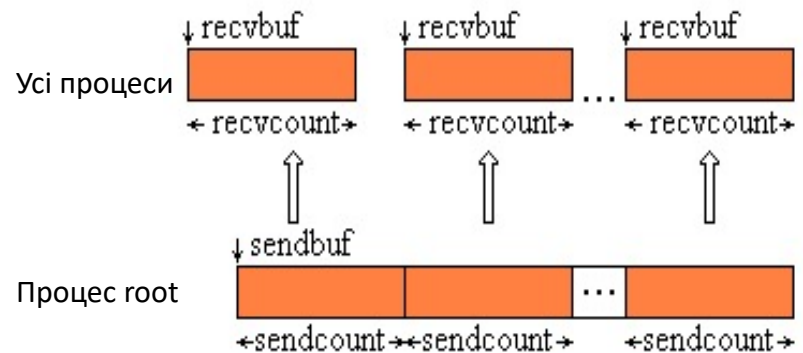
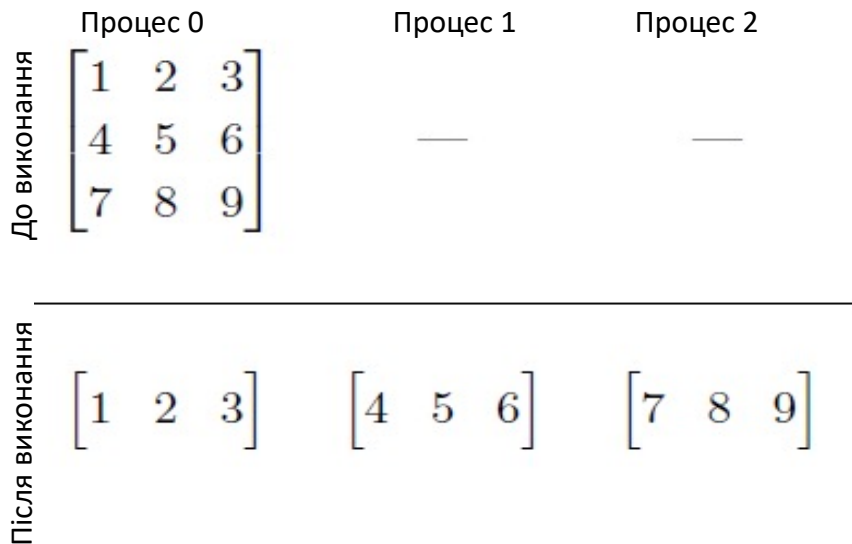
Приклад:

```
/*MASTER*/
```

```
MPI_Scatter(    MA, k*N, MPI_INT,  
                MAh, k*N, MPI_INT,  
                0, MPI_COMM_WORLD);
```

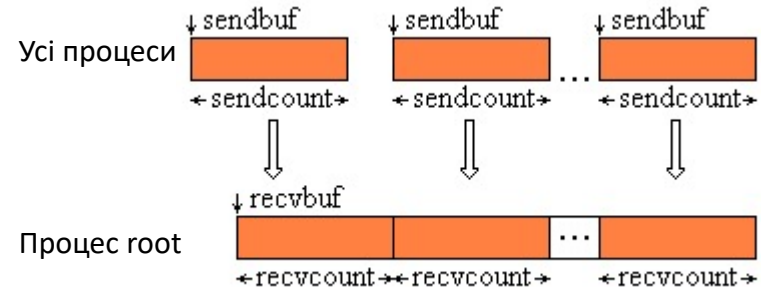
```
/*WORKER*/
```

```
MPI_Scatter(    NULL, 0, MPI_DATATYPE_NULL,  
                MAh, k*N, MPI_INT,  
                0, MPI_COMM_WORLD);
```



Метод MPI_Gather

```
MPI_Gather(  
    void *sendbuf,  
    int sendcnt,  
    MPI_Datatype sendtype,  
    void *recvbuf, int recvcnt,  
    MPI_Datatype recvtype,  
    int root, MPI_Comm comm);
```



- Параметри `recvbuf`, `recvcnt`, `recvtype` сприймаються тільки процесом-отримувачем, інші процеси ігнорують ці значення. Буфер відправки в кожному процесі має бути розміром не менше за `sendcnt`, а буфер отримувача даних має бути не менше за `recvcnt * P`, де P - кількість процесів.
- Метод може використовувати один і той самий буфер як буфер відправки та прийому у відправляючій задачі. Для цього необхідно аргумент `sendbuf` встановити рівним константі `MPI_IN_PLACE`. При цьому аргументи `sendcnt` та `sendtype` ігноруються. Бажано дотримуватися попередніх рекомендацій щодо встановлення значень аргументів, які ігноруються.

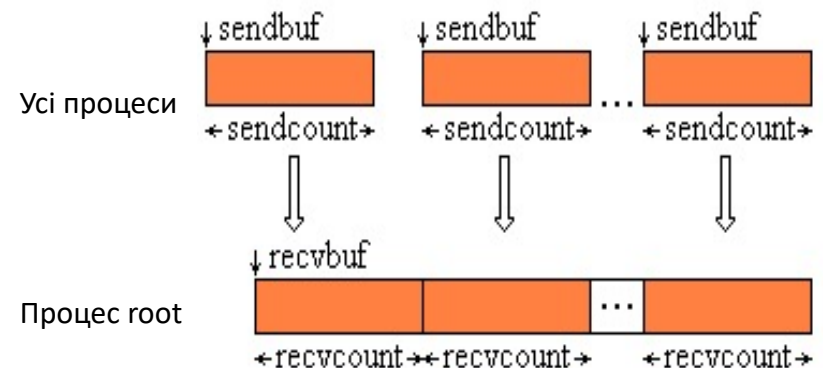
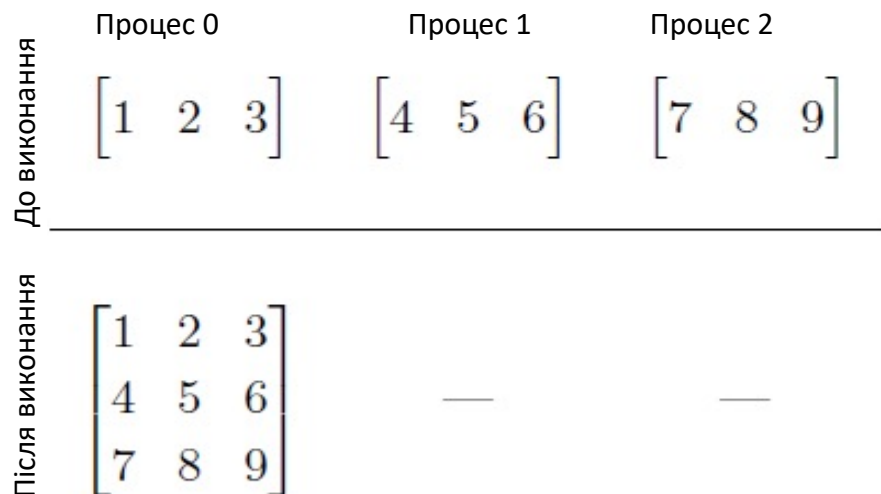
Приклад:

```
/*MASTER*/
```

```
MPI_Gather( MAh, k*N, MPI_INT,  
            MR, k*N, MPI_INT,  
            0, MPI_COMM_WORLD);
```

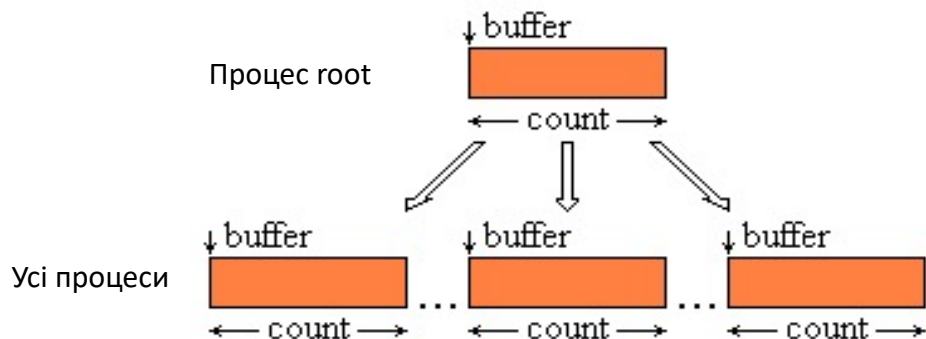
```
/*WORKER*/
```

```
MPI_Gather( MAh, k*N, MPI_INT,  
            NULL, 0, MPI_INT,  
            0, MPI_COMM_WORLD);
```



Метод MPI_Bcast

```
MPI_Bcast ( void *buffer,  
            int count,  
            MPI_Datatype type,  
            int root,  
            MPI_Comm comm)
```



Приклад:

```
MPI_Bcast ( MR, N*N, MPI_INT,  
           1, MPI_COMM_WORLD) ;
```

Процес 0

Процес 1

Процес 2

До виконання

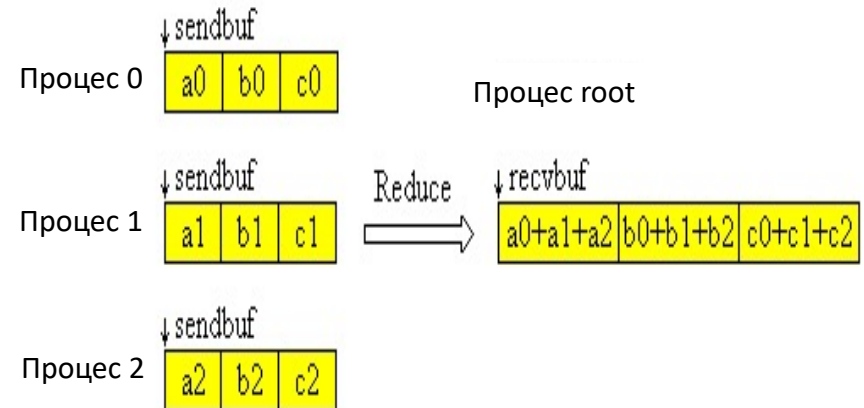
$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Після виконання

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$
$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$
$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Метод MPI_Reduce

MPI_Reduce (**void** *sendbuf,
void *recvbuf,
int count,
MPI_Datatype datatype,
MPI_Op op,
int root,
int comm)



Операції:

MPI_MAX	Максимум	int, float
MPI_MIN	Мінімум	int, float
MPI_SUM	Сума	int, float
MPI_PROD	Добуток	int, float
MPI_LAND	Логічне «І»	int
MPI_BAND	Побітове «І»	int
MPI_LOR	Логічне «АБО»	int
MPI_BOR	Побітове «АБО»	int
MPI_LXOR	Логічне виключне «АБО»	int
MPI_BXOR	Побітове виключне «АБО»	int

Приклад:

```
/*MASTER*/  
MPI_Reduce( MA, MR, N*N,  
            MPI_INT, MPI_SUM,  
            0, MPI_COMM_WORLD) ;  
  
/*WORKER*/  
MPI_Reduce( MA, NULL, N*N,  
            MPI_INT, MPI_SUM,  
            0, MPI_COMM_WORLD) ;
```

	Процес 0	Процес 1	Процес 2
До виконання	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$	$\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$	$\begin{bmatrix} 9 & 10 \\ 11 & 12 \end{bmatrix}$
Після виконання	$\begin{bmatrix} 15 & 18 \\ 21 & 24 \end{bmatrix}$		

Метод MPI_AllReduce

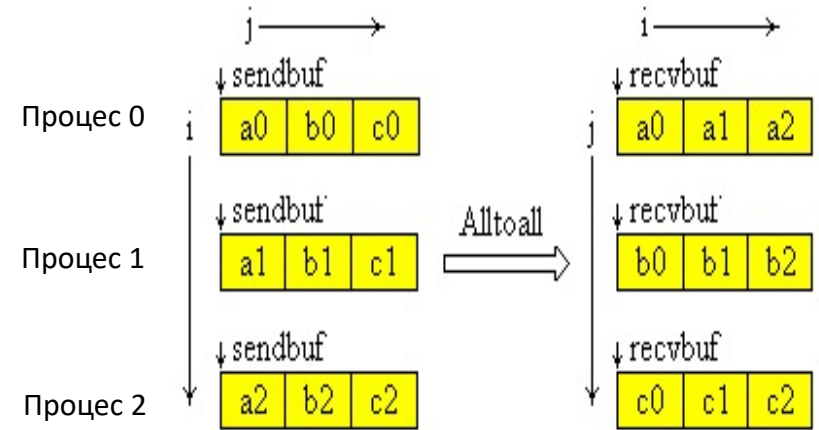
```
MPI_Allreduce(    void *sendbuf,  
                  void *recvbuf,  
                  int count,  
                  MPI_Datatype datatype,  
                  MPI_op op,  
                  MPI_Comm comm);
```

Приклад: MPI_Allreduce(MA, MR,
 N*N, MPI_INT,
 MPI_SUM, MPI_COMM_WORLD);

	Процес 0	Процес 1	Процес 2
До виконання	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$	$\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$	$\begin{bmatrix} 9 & 10 \\ 11 & 12 \end{bmatrix}$
Після виконання	$\begin{bmatrix} 15 & 18 \\ 21 & 24 \end{bmatrix}$	$\begin{bmatrix} 15 & 18 \\ 21 & 24 \end{bmatrix}$	$\begin{bmatrix} 15 & 18 \\ 21 & 24 \end{bmatrix}$

Метод MPI_Alltoall

```
MPI_Alltoall (void *sendbuf,
              int sendcount,
              MPI_Datatype sendtype,
              void *recvbuf,
              int recvcount,
              MPI_Datatype recvtype,
              MPI_Comm comm);
```



i -ий процес відправляє дані у i -ий елемент буфера-отримувача

Приклад: MPI_Alltoall(MPI_IN_PLACE, 0, MPI_DATATYPE_NULL, MA, 1, MPI_INT, MPI_COMM_WORLD);

До виконання

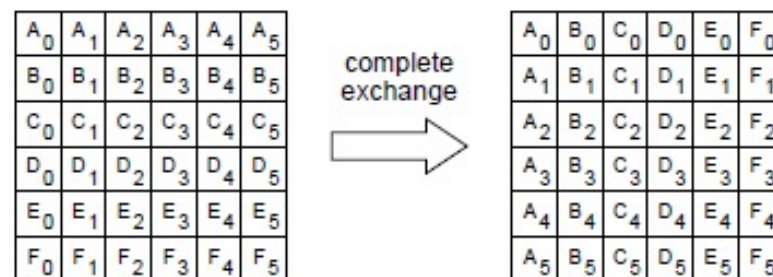
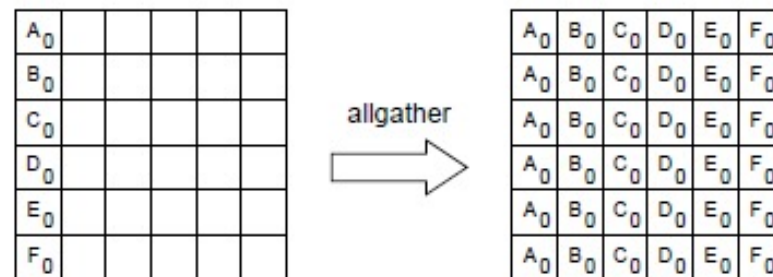
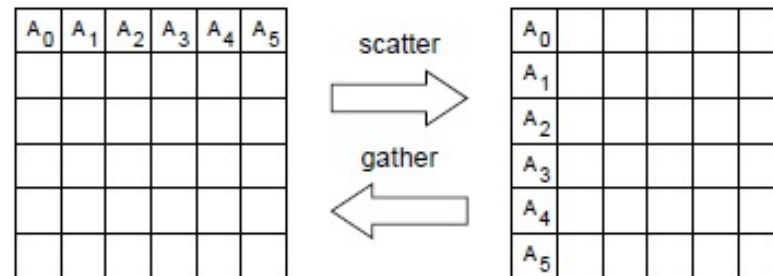
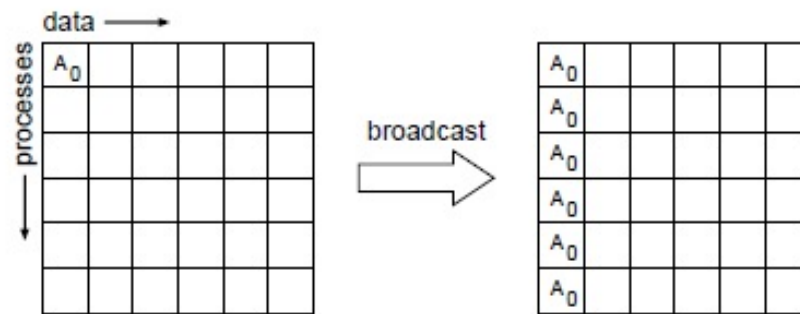
Процес 0	Процес 1	Процес 2
[1 2 3]	[4 5 6]	[7 8 9]

Після виконання

[1 4 7]	[2 5 8]	[4 6 9]
---------	---------	---------

Інші методи

- Методи, які містять вказівку - аргумент `int *displs` - на зміщення фрагментів масивів, що обробляються в процесах (тільки root-процес його обов'язково вказує, інші процеси ігнорують цей аргумент)
 - MPI_Gatherv
 - MPI_Scatterv
 - MPI_Allgatherv
 - MPI_Alltoallv



Example 1

Усі приклади наведені з опису стандарту MPI [<http://hpc.inf.ethz.ch/sec/nbc-proposal-11-26-2008-2.pdf>]

Gather 100 ints from every process in group to root.

```
MPI_Comm comm;
int gsize, sendarray[100];
int root, *rbuf;
...
MPI_Comm_size(comm, &gsize);
rbuf = (int *)malloc(gsize*100*sizeof(int));
MPI_Gather(sendarray, 100, MPI_INT,
            rbuf, 100, MPI_INT,
            root, comm);
```


Example 2

Previous example modified | only the root allocates memory for the receive buffer.

```
MPI_Comm comm;
int gsize, sendarray[100];
int root, myrank, *rbuf;
...
MPI_Comm_rank(comm, &myrank);
if (myrank == root) {
    MPI_Comm_size(comm, &gsize);
    rbuf = (int *)malloc(gsize*100*sizeof(int));
}
MPI_Gather(sendarray, 100, MPI_INT,
           rbuf, 100, MPI_INT,
           root, comm);
```

Example 3

Do the same as the previous example, but use a derived datatype. Note that the type cannot be the entire set of $gsize \times 100$ ints since type matching is defined pairwise between the root and each process in the gather.

```
MPI_Comm comm;
int gsize, sendarray[100];
int root, *rbuf;
MPI_Datatype rtype;
...
MPI_Comm_size(comm, &gsize);
MPI_Type_contiguous(100, MPI_INT, &rtype);
MPI_Type_commit(&rtype);
rbuf = (int *)malloc(gsize*100*sizeof(int));
MPI_Gather(sendarray, 100, MPI_INT,
           rbuf, 1, rtype, root, comm);
```

Example 4

Now have each process send 100 ints to root, but place each set (of 100) stride ints apart at receiving end. Use MPI_GATHERV and the displs argument to achieve this effect. Assume $\text{stride} \geq 100$.

```
MPI_Comm comm;
int gsize, sendarray[100];
int root, *rbuf, stride;
int *displs, i, *rcounts;
...
MPI_Comm_size(comm, &gsize);
rbuf = (int *)malloc(gsize*stride*sizeof(int));
displs = (int *)malloc(gsize*sizeof(int));
rcounts = (int *)malloc(gsize*sizeof(int));
for (i=0; i<gsize; ++i) {
    displs[i] = i*stride;
    rcounts[i] = 100;
}
MPI_Gatherv(sendarray, 100, MPI_INT, rbuf, rcounts, displs,
MPI_INT,
root, comm);
```

Example 5

Same as Example 5.5 on the receiving side, but send the 100 ints from the 0th column of a 100x150 int array, in C.

```
MPI_Comm comm;
int gsize, sendarray[100][150];
int root, *rbuf, stride;
MPI_Datatype stype;
int *displs, i, *rcounts;
...
MPI_Comm_size(comm, &gsize);
rbuf = (int *)malloc(gsize*stride*sizeof(int));
displs = (int *)malloc(gsize*sizeof(int));
rcounts = (int *)malloc(gsize*sizeof(int));
for (i=0; i<gsize; ++i) {
    displs[i] = i*stride;
    rcounts[i] = 100;
}
/* Create datatype for 1 column of array */
MPI_Type_vector(100, 1, 150, MPI_INT, &stype);
MPI_Type_commit(&stype);
MPI_Gatherv(sendarray, 1, stype, rbuf, rcounts, displs, MPI_INT,
root, comm);
```

Example 6

Process i sends $(100-i)$ ints from the i -th column of a 100×150 int array, in C. It is received into a buffer with stride, as in the previous two examples.

```
MPI_Comm comm;
int gsize, sendarray[100][150], *sptr;
int root, *rbuf, stride, myrank;
MPI_Datatype stype;
int *displs, i, *rcounts;
...
MPI_Comm_size(comm, &gsize);
MPI_Comm_rank(comm, &myrank);
rbuf = (int *)malloc(gsize*stride*sizeof(int));
displs = (int *)malloc(gsize*sizeof(int));
rcounts = (int *)malloc(gsize*sizeof(int));
for (i=0; i<gsize; ++i) {
    displs[i] = i*stride;
    rcounts[i] = 100-i; /* note change from previous example */
}
/* Create datatype for the column we are sending */
MPI_Type_vector(100-myrank, 1, 150, MPI_INT, &stype);
MPI_Type_commit(&stype);
/* sptr is the address of start of "myrank" column
*/
sptr = &sendarray[0][myrank];
MPI_Gatherv(sptr, 1, stype, rbuf, rcounts, displs, MPI_INT, root, comm);
```

Example 7

The reverse of Example 5.2. Scatter sets of 100 ints from the root to each process in the group.

```
MPI_Comm comm;
int gsize, *sendbuf;
int root, rbuf[100];
...
MPI_Comm_size(comm, &gsize);
sendbuf = (int *)malloc(gsize*100*sizeof(int));
...
MPI_Scatter(sendbuf, 100, MPI_INT, rbuf, 100,
MPI_INT, root, comm);
```

Example 8

The starting order of collective operations on a particular communicator defines their matching. The following example shows an erroneous matching of different collective operations on the same communicator

```
MPI_Request req;
switch(rank) {
    case 0:
        /* erroneous matching */
        MPI_Ibarrier(comm, &req);
        MPI_Bcast(buf1, count, type, 0, comm);
        MPI_Wait(&req, MPI_STATUS_IGNORE);
        break;
    case 1:
        /* erroneous matching */
        MPI_Bcast(buf1, count, type, 0, comm);
        MPI_Ibarrier(comm, &req);
        MPI_Wait(&req, MPI_STATUS_IGNORE);
        break;
}
```

Example 9

Collective and point-to-point requests can be mixed in functions that enable multiple completions. If started with two processes, the following program is valid

```
MPI_Request reqs[2];
switch(rank) {
    case 0:
        MPI_Ibarrier(comm, &reqs[0]);
        MPI_Send(buf, count, dtype, 1, tag, comm);
        MPI_Wait(&reqs[0], MPI_STATUS_IGNORE);
        break;
    case 1:
        MPI_Irecv(buf, count, dtype, 0, tag, comm, &reqs[0]);
        MPI_Ibarrier(comm, &reqs[1]);
        MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);
        break;
}
```


Example 10

Multiple nonblocking collective operations can be outstanding on a single communicator and match in order

```
MPI_Request reqs[3];
compute(buf1);
MPI_Ibcast(buf1, count, type, 0, comm, &reqs[0]);
compute(buf2);
MPI_Ibcast(buf2, count, type, 0, comm, &reqs[1]);
compute(buf3);
MPI_Ibcast(buf3, count, type, 0, comm, &reqs[2]);
MPI_Waitall(3, reqs, MPI_STATUSES_IGNORE);
```

Current Practice #2

```
int main(int argc, char *argv[]){
    int me, count;
    void *data;
    ...
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &me);
    if(me == 0){
        /* get input, create buffer ``data'' */
        ...
    }
    MPI_Bcast(data, count, MPI_BYTE, 0, MPI_COMM_WORLD);
    ...
    MPI_Finalize();
    return 0;
}
```

Example. Create group of processors

```
int main(int argc, char *argv[]){
    int me, count, count2;
    void *send_buf, *recv_buf, *send_buf2, *re
    MPI_Group group_world, grprem;
    MPI_Comm commslave;
    static int ranks[] = {0};
    ...
    MPI_Init(&argc, &argv);
    MPI_Comm_group(MPI_COMM_WORLD, &group_world);
    MPI_Comm_rank(MPI_COMM_WORLD, &me); /* local */
    MPI_Group_excl(group_world, 1, ranks, &grprem); /* local */
    MPI_Comm_create(MPI_COMM_WORLD, grprem, &commslave);
    if(me != 0) {
        /* compute on slave */
        ...
        MPI_Reduce(send_buf,recv_buf,count, MPI_INT, MPI_SUM, 1, commslave);
        ...
        MPI_Comm_free(&commslave);
    }
    /* zero falls through immediately to this reduce, others do later... */
    MPI_Reduce(send_buf2, recv_buf2, count2, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Group_free(&group_world);
    MPI_Group_free(&grprem);
    MPI_Finalize();
    return 0;
}
```

Example #4

```
#define TAG_ARBITRARY 12345
#define SOME_COUNT 50
int main(int argc, char *argv[]) {
    int me;
    MPI_Request request[2];
    MPI_Status status[2];
    MPI_Group group_world, subgroup;
    int ranks[] = {2, 4, 6, 8};
    MPI_Comm the_comm;
    ...
    MPI_Init(&argc, &argv);
    MPI_Comm_group(MPI_COMM_WORLD, &group_world);
    MPI_Group_incl(group_world, 4, ranks, &subgroup); /* local */
    MPI_Group_rank(subgroup, &me); /* local */
    MPI_Comm_create(MPI_COMM_WORLD, subgroup, &the_comm);
    if(me != MPI_UNDEFINED){
        MPI_Irecv(buff1, count, MPI_DOUBLE, MPI_ANY_SOURCE, TAG_ARBITRARY,
            the_comm, request);
        MPI_Isend(buff2, count, MPI_DOUBLE, (me+1)%4, TAG_ARBITRARY,
            the_comm, request+1);
        for(i = 0; i < SOME_COUNT; i++)
            MPI_Reduce(..., the_comm);
        MPI_Waitall(2, request, status);
        MPI_Comm_free(&the_comm);
    }
    MPI_Group_free(&group_world);
    MPI_Group_free(&subgroup);
    MPI_Finalize();
    return 0;
}
```

!!! Передивіться приклади з опису стандарту
<http://htor.inf.ethz.ch/sec/nbc-proposal-11-26-2008-2.pdf>

!!! При виконанні курсової роботи зверніть увагу на використання колективних методів та неблокуючих колективних методів, а також на використання неблокуючих методів.

!!! Найпростіші приклади можна оглянути тут
<https://cvw.cac.cornell.edu/mpicc/data-movement/allgather>